

C 語言 Socket 程式設計: 建構支援 Python Requests 存取之高效能 HTTP API 伺服器深度解析

在現代分散式系統與微服務架構中，應用程式介面 (API) 扮演著異質系統間溝通的關鍵樞紐。儘管當前的高階語言 (如 Python、Go、Node.js) 提供了極度封裝且高度易用的網路模組，使開發者能以寥寥數行程式碼建構網頁伺服器，但要深刻理解網際網路協定套件 (TCP/IP) 的底層運作機制、記憶體管理、以及作業系統核心 (Kernel) 如何處理網路 I/O，直接使用 C 語言與 POSIX 套接字 (Socket) 系統呼叫 (System Calls) 從零開始構建 HTTP 伺服器，仍是無可取代的專業實踐¹。本研究報告旨在提供一份窮盡細節的技術解析，詳細探討如何使用 C 語言編寫一個能精準接收 Python requests.get 與 requests.post 請求，並正確回傳 JSON 或純文字格式資料的 HTTP API 伺服器。報告將深入剖析每一步驟的邏輯、底層系統呼叫機制、巨集 (Macro) 與結構體 (Structs) 的記憶體佈局，以及各類常數與整數的確切作業系統意義。此外，本報告將廣泛探討 TCP 字節流 (Byte Stream) 無邊界特性的實務處理、HTTP 協定解析狀態機的實作，以及如何防禦諸如 SIGPIPE 訊號等系統層級例外，從而建構具備高度穩定性與可靠性的網路基礎設施。

一、網路底層巨集與常數之整數意義與核心機制解析

在 C 語言的網路程式設計中，開發者頻繁使用由 POSIX 標準定義、並宣告於 <sys/socket.h> 與 <netinet/in.h> 等標準標頭檔中的巨集 (Macros)¹。這些巨集在編譯階段 (Compilation Phase) 會被預處理器替換為特定的整數值。作業系統核心的網路堆疊 (Network Stack) 正是藉由識別這些整數值，來決定網路資源的分配策略、記憶體佈局與資料傳輸行為⁵。

巨集名稱	對應整數值	核心意義與底層作業系統操作解析
AF_INET	2	代表 Address Family Internet。此數值指示作業系統核心建立一個使用 IPv4 網際網路協定家族的通訊端點。核心據此決定為套接字分配 32 位元長度的 IP 地址空間與對應的路由邏輯 ² 。
SOCK_STREAM	1	代表套接字類型為資料流 (Stream Socket)。在 AF_INET 家族的脈絡下，此值指示核心使用傳輸控制協定 (TCP)。核心將為此通訊端點配置發送與接收緩衝區

		，並啟動提供可靠、連線導向且無資料邊界的雙向字節流保證機制 ² 。
SOL_SOCKET	1	代表 Socket Option Level, 數值 1 意指「套接字層級」。當呼叫 setsockopt 進行參數微調時，此值指示核心將該設定應用於套接字物件本身（如位址重用、保活機制），而非應用於更底層的 TCP (IPPROTO_TCP) 或 IP 協定層 ⁸ 。
SO_REUSEADDR	2	允許套接字強制綁定至處於 TIME_WAIT 狀態的本地網路地址與通訊埠。在 TCP 協定規範中，主動關閉連線的一方其通訊埠會進入數分鐘的等待期，設定此選項為真（非零整數）可避開「地址已在使用中 (EADDRINUSE)」的錯誤，對伺服器頻繁重啟至關重要 ⁸ 。
INADDR_ANY	0	對應整數 0（即位址 0.0.0.0）。這是一個萬用字元（Wildcard）位址，指示伺服器綁定並監聽本機端所有可用的網路介面（Network Interfaces）。無論客戶端是透過回環地址（Loopback, 127.0.0.1）還是外部實體網卡 IP 發起請求，伺服器皆能無差別接收 ² 。
SIGPIPE	13	POSIX 標準定義的系統訊號（Signal）。當處理程序嘗試使用 send() 或 write() 將資料推入一個接收端已經異常關閉（或發送了 RST 封包）的連線時，核心會向該程序發送

		編號 13 的訊號。該訊號的預設行為是強行終止處理程序 (Process Termination) ⁵ 。
--	--	--

深入理解上述整數值的意義，不僅有助於在跨平台移植或除錯時快速定位問題，更能揭示 C 語言系統呼叫的本質：應用程式不過是將特定的數值與記憶體指標，透過系統呼叫介面傳遞給作業系統核心，由核心根據中斷向量表 (Interrupt Vector Table) 執行對應的網路封包封裝與解封裝作業。

二、網路位址結構體 (Structs) 之記憶體佈局與轉型多型性

在呼叫任何涉及綁定或連線的網路函數之前，應用程式必須先建構並初始化位址結構體。C 語言利用結構體來嚴密封裝 IP 位址與通訊埠資訊。由於 C 語言本身不具備現代物件導向語言的繼承 (Inheritance) 機制，POSIX 標準採用了特殊的指標強制轉型 (Pointer Casting) 技術來達成多型 (Polymorphism) 的效果，允許單一系統呼叫處理多種不同的網路協定位址¹⁴。

1. 泛型位址結構體：struct sockaddr

struct sockaddr 是系統呼叫 (如 bind、accept、connect) 所預期的標準參數型別¹⁶。可以將其視為一個 16 位元組 (Bytes) 的泛型資料容器，其唯一的目的是告訴核心該如何讀取後續的記憶體區塊¹⁴。

其標準記憶體佈局如下：

- **sa_family_t sa_family**：佔用 2 位元組的無號整數，用於儲存位址家族標識 (例如 AF_INET)。
- **char sa_data[14]**：佔用 14 位元組的連續字元陣列，作為儲存協定專屬位址資料 (如 IP 與通訊埠) 的原始位元組空間¹⁴。

2. IPv4 專屬位址結構體：struct sockaddr_in

直接操作 sa_data 陣列來寫入 IP 與通訊埠極為困難且容易出錯。因此，標準函式庫提供了 struct sockaddr_in (in 代表 Internet)，為開發者提供了清晰命名的欄位，精確對應 IPv4 的需求，同時保證其總記憶體大小與 struct sockaddr 完美對齊 (同為 16 位元組)⁴。

struct sockaddr_in 的內部欄位與位元組意義解析：

- **sa_family_t sin_family** (2 位元組)：必須設定為 AF_INET (數值 2)。當此結構體被強制轉型為 struct sockaddr 傳遞給核心時，核心首先讀取這兩個位元組，一旦確認數值為 2，核心網路堆疊就會將後續的 14 個位元組依照 IPv4 的規則進行解碼⁴。
- **in_port_t sin_port** (2 位元組)：代表 16 位元的通訊埠號 (範圍 0 至 65535)。網路通訊協定統一規定使用「大端序 (Big-Endian)」作為網路字節序。因此，開發者必須使用 htons() (Host to Network Short) 函數，將本機系統的字節序 (多數現代 CPU 為小端序 Little-Endian) 轉換為網路字節序。例如，若將通訊埠設為 8080，未經轉換直接寫入將導致核心解析出完全不同的通訊埠號⁴。
- **struct in_addr sin_addr** (4 位元組)：內部僅包含一個 32 位元的無號整數 s_addr，代表 IPv4 位址。在建構 API 伺服器時，通常將其賦值為 INADDR_ANY (數值 0)，指示綁定所有網卡。若需指定特定 IP，可使用 inet_pton() 將字串轉換為二進位網路字節序，或使用 htonl() 處理整數⁴。

- **unsigned char sin_zero[8]** (8 位元組)：這是為了確保 struct sockaddr_in 的總大小達到 16 位元組而存在的填充空間 (Padding)。此欄位沒有任何實質網路意義，但在實務上，開發者必須使用 `memset(&address, 0, sizeof(address))` 將整個結構體初始化為零，以確保這 8 個位元組皆為空值，避免核心讀取到不可預期的垃圾記憶體資料⁴。

三、POSIX Socket 系統呼叫與伺服器生命週期深度解剖

要讓 Python 的 `requests.get` 或 `requests.post` 能夠成功將 HTTP 請求打到 C 語言編寫的端點並取得回傳值，C 伺服器必須嚴謹地依照 TCP/IP 協定的規範，完整經歷通訊端點建立、綁定、監聽與接受連線的生命週期。每一階段皆對應特定的系統呼叫，操作著作業系統核心內部的檔案描述符 (File Descriptor)、雜湊表與狀態佇列。

第一步：建立套接字端點 `socket()`

```
C
int server_fd = socket(AF_INET, SOCK_STREAM, 0)
if (server_fd == 0)
    perror("socket failed")
    exit(EXIT_FAILURE)
```

邏輯與底層操作：`socket()` 系統呼叫指示作業系統的虛擬檔案系統 (VFS) 與網路子系統，在核心記憶體中分配一個新的通訊端點資料結構 (Socket Control Block)。

- 參數 `domain(AF_INET)` 與 `type(SOCK_STREAM)` 告知核心這是一個 IPv4 的 TCP 連線⁷。
- 參數 `protocol` 設為 0 讓核心自動選擇對應該 `domain` 與 `type` 的預設協定 (即 TCP)⁷。
- 核心成功配置資源後，會回傳一個非負整數的檔案描述符 (File Descriptor, FD)，應用程式後續的所有網路操作皆透過此整數控制把柄 (Handle) 進行。若回傳 -1，表示系統檔案描述符已耗盡 (如 `EMFILE` 或 `ENFILE`)，或權限不足¹⁶。

第二步：設定位址重用選項 `setsockopt()`

```
C
int opt = 1
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0)
```

```

    perror("setsockopt failed")
    exit(EXIT_FAILURE)
}

```

邏輯與底層操作：在 API 伺服器的開發與部署過程中，程式的重啟是家常便飯。根據 TCP 狀態機規範，當伺服器主動中斷連線時，該套接字會進入 TIME_WAIT 狀態數分鐘，確保網路上延遲的封包自然消亡¹¹。若不設定 SO_REUSEADDR，立即重啟伺服器並嘗試綁定相同通訊埠時，bind() 會遭遇「位址已被佔用 (Address already in use)」的錯誤⁸。將此選項值設為 1 (整數，長度透過 sizeof(opt) 傳遞)，能指示核心放行處於 TIME_WAIT 狀態的位址綁定請求¹⁹。

第三步：綁定位址與通訊埠 bind()

```

C
struct sockaddr_in address;
memset(&address, 0, sizeof address);
address.sin_family = AF_INET;
address.sin_addr.s_addr = htonl(INADDR_ANY);
address.sin_port = htons(8080);

if (bind(server_fd, (struct sockaddr *)&address, sizeof address) < 0)
    perror("bind failed")
    exit(EXIT_FAILURE)
}

```

邏輯與底層操作：bind() 系統呼叫的作用是将核心中的套接字物件與特定的本機 IP 位址和通訊埠號建立關聯 (登記至核心的 TCP 監聽雜湊表中)。

- 呼叫時，必須使用 (struct sockaddr *) 對 &address 進行強制轉型。這是因為 bind 函式簽章設計為接受泛型位址結構¹⁴。
- 第三個參數 sizeof(address) 告訴核心該讀取多少位元組的記憶體，防止越界存取 (Buffer Over-read)¹⁷。

第四步：啟動被動監聽 listen()

```

C
if (listen(server_fd, SOMAXCONN) < 0)
    perror("listen failed")
    exit(EXIT_FAILURE)

```

邏輯與底層操作: `listen()` 將該套接字由主動連線模式(預設)轉換為被動監聽模式。此時, 伺服器正式具備接收 TCP 三方交握(Three-way Handshake)的能力¹⁰。

- 參數 `backlog` (此處使用系統巨集 `SOMAXCONN`, 通常為 128 或 4096) 定義了核心維護的「已完成 TCP 交握但尚未被應用程式 `accept()` 取走」的完全建立連線佇列(Completely Established Queue)的最大長度¹⁰。當 Python 客戶端並發大量 `requests.get()` 且伺服器處理不及時, 超出的連線請求會被核心拒絕或忽略, 導致客戶端拋出連線超時或重置異常¹⁰。

第五步: 接受客戶端連線 `accept()`

```

C
struct sockaddr_in client_addr;
socklen_t addrlen = sizeof client_addr;
int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addrlen);
if (client_fd < 0)
    perror("accept failed")
    // 通常在此進行錯誤記錄並 continue, 而非直接退出伺服器

```

邏輯與底層操作: `accept()` 是伺服器主迴圈的入口, 它是一個阻塞型(Blocking)的系統呼叫。

- 當連線佇列為空時, 呼叫 `accept()` 的執行緒會被作業系統排程器置於休眠狀態(Sleep), 釋放 CPU 資源¹⁶。
- 當 Python 客戶端的 `requests` 模組底層完成 TCP 連線建立後, `accept()` 會被喚醒, 從佇列中取出該連線, 並由核心動態建立一個全新的檔案描述符(`client_fd`)¹⁶。
- 原有的 `server_fd` 繼續負責監聽其他新進連線, 而新產生的 `client_fd` 則專門用於與該特定 Python 客戶端進行全雙工(Full-duplex)的資料收發¹⁶。客戶端的 IP 與通訊埠資訊會被核心回填至傳入的 `client_addr` 記憶體區塊中¹⁶。

四、克服 TCP 字節流特性與 HTTP 協定邊界解析之挑戰

當成功獲取 `client_fd` 後, 伺服器程式碼必須讀取 Python 客戶端發送過來的 HTTP 請求字串。此階段是 C 語言網路程式設計中最容易發生錯誤的環節, 核心挑戰在於開發者往往誤將 TCP 視為

「訊息導向 (Message-Oriented)」的協定，忽略了其字節流的本質。

1. TCP 邊界缺失與 `recv()` 呼叫的陷阱

TCP 是一種提供可靠傳輸的字節流協定 (Byte-Stream Protocol)⁷。它保證資料抵達的順序與正確性，但完全不保留應用層的訊息邊界²³。當 Python 的 `requests.post()` 傳送了一個包含 2000 位元組 JSON Body 的 HTTP 請求時，這 2000 多個位元組在網路傳輸過程中，可能因為最大傳輸單元 (MTU)、TCP 壅塞控制視窗大小或網路延遲，被底層網路切分為多次封包抵達伺服器²⁵。

在 C 語言中，讀取資料的系統呼叫為 `recv()`：

```
C
ssize_t recv(int sockfd, void *buf, size_t len, int flags)
```

`recv()` 的行為模式如下：它會檢查核心為該套接字維護的接收緩衝區²⁴。如果緩衝區內有資料，`recv()` 會將其複製到應用層的 `buf` 中，並回傳實際複製的位元組數³。

- 若要求讀取 4096 位元組 (`len = 4096`)，但核心目前只接收到 800 位元組，`recv()` 會立刻回傳 800，而不會阻塞等待剩餘資料²⁶。
- 若回傳 0，代表客戶端已正常發送 FIN 封包關閉連線 (EOF)²⁷。
- 若回傳 -1，代表發生系統級錯誤，需進一步檢查 `errno`²⁷。

如果 C 伺服器只執行一次單純的 `recv()`，可能只會讀取到一半的 HTTP 標頭，或者讀取到完整的標頭加上一半的 JSON Body。後續將這半截資料送入 JSON 解析器必定導致失敗。因此，應用層必須自行實作緩衝區管理與狀態機 (State Machine) 來拼湊完整的訊息。

2. 解析 HTTP 請求結構與標頭

HTTP 請求基於 ASCII/UTF-8 的純文字協定，其結構具有嚴格的規範，主要分為三大部分：

1. 起始行 (**Start Line**)：包含方法、URI 與協定版本，以回車換行符號 `\r\n` 結尾。如 `POST /api/data HTTP/1.1\r\n`²⁸。
2. 標頭欄位 (**Headers**)：包含多行 `Key: Value\r\n`，用以傳遞詮釋資料 (Metadata)。Python `requests` 會自動帶入 `User-Agent`、`Accept`、`Accept-Encoding` 等標頭²⁸。標頭區塊的結束，絕對標誌為連續的兩個回車換行符號 `\r\n\r\n`³。
3. 主體 (**Message Body**)：緊跟在 `\r\n\r\n` 之後的二進位或文字資料。若是 GET 請求通常無主體；若是 POST 請求，主體的確切位元組長度由標頭中的 `Content-Length` 指定³。

3. 動態緩衝區讀取與 `Content-Length` 提取邏輯

為了穩健地處理 Python 發送的 `requests.post`，C 伺服器必須實作以下讀取與解析邏輯，每一步的底層操作都需極度謹慎以避免記憶體錯誤：

步驟一：尋找 HTTP 標頭邊界 伺服器應在一個迴圈中呼叫 `recv()`，將資料附加到動態配置 (使用

malloc 或 realloc) 或足夠大的靜態陣列中。每次接收新資料後，利用標準函式庫的 strstr() 搜尋 \r\n\r\n 模式²⁵。

- char *strstr(const char *haystack, const char *needle); 函式會在記憶體中逐位元組比對，回傳指向第一個匹配子字串的記憶體位址指標，若未找到則回傳 NULL³³。
- 一旦 strstr 回傳非空指標，即代表伺服器已將完整的 HTTP 標頭讀入記憶體中³¹。

步驟二：解析 **Content-Length** 提取 **Payload** 長度 在確認標頭完整後，必須判斷這是否為帶有資料主體的 POST 請求。伺服器再次使用 strstr 在已接收的字串中尋找 Content-Length: 子字串³。若存在，則透過 sscanf() 進行字串到整數的格式化掃描與提取³⁶。

```
C
char *cl_ptr = strstr(request_buffer, "Content-Length:");
int content_length = 0;
if (cl_ptr)
    // 提取標頭後面的整數值
    if (sscanf(cl_ptr, "Content-Length: %d", &content_length) != 1)
        // 錯誤處理：無法正確解析整數
```

- sscanf() 是處理格式化文字輸入的強大工具，它依照提供的格式字串 (Format String) 嘗試匹配並轉換資料型別，回傳成功轉換的變數數量³⁶。在此例中，它會略過空格並讀取整數值寫入 content_length 的記憶體位址中³⁶。

步驟三：確保主體資料 (**Body**) 接收完整 這是解決 recv() 提前回傳的核心邏輯。標頭的結尾指標加上 4 個位元組 (\r\n\r\n 的長度) 即為主體資料在緩衝區中的起始記憶體位址。伺服器必須計算：已接收的主體位元組數 = 總共 recv() 的位元組數 - 標頭的總長度²⁶。

如果 已接收的主體位元組數 < content_length，伺服器必須再次進入一個專門接收 Body 的 while 迴圈，持續呼叫 recv()，直到拼湊滿 content_length 個位元組為止²⁶。只有在滿足此條件後，方可將 Body 資料交由應用層的 JSON 解析庫 (如 cJSON) 進行後續的業務邏輯運算。

五、Python Requests 之特徵與 HTTP 回應建構 (Response Framing)

Python 的 requests 函式庫以其「供人類使用 (HTTP for Humans)」的設計理念聞名。當開發者在 Python 端執行 requests.post('http://server/', json={'key': 'value'}) 時，該函式庫不僅自動將字典序列化為 JSON 字串，還會自動在底層建構符合規範的 HTTP 標頭，包括自動加上 Content-Type: application/json，精確計算 JSON 字串長度並產生 Content-Length，同時附加預設的 User-Agent 與 Accept-Encoding 標頭³⁰。

相對地，當 Python 接收 C 伺服器的回應時，它的行為完全依賴於伺服器所建構的 HTTP 協定格

式⁴⁰。如果 C 伺服器僅僅傳回 JSON 字串而沒有合法的 HTTP 標頭，Python 客戶端底層的 urllib3 模組將無法解析，進而拋出 ProtocolError 或 ConnectionError 例外。

1. 建構合法的 HTTP 回應

為了讓 Python 的 `response = requests.get()` 能夠正確取得資料，並且讓 `response.json()` 函數成功將字串反序列化為 Python 字典，C 伺服器必須嚴格按照以下格式組合字串³⁹：

```
HTTP
HTTP/1.1 200 OK\r\n
Content-Type: application/json; charset=utf-8\r\n
Content-Length: 21\r\n
Connection: close\r\n
\r\n
{"status":"success","data":123}
```

在 C 語言中，字串的拼接必須極度小心以避免緩衝區溢位。最佳實踐是使用 `snprintf()` 函數，它強制要求指定目標緩衝區的最大長度，有效防堵了記憶體越界寫入的安全漏洞³⁶。

```
C
const char json_payload = "{\"status\":\"success\",\"data\":123}"
int payload_len = strlen(json_payload);
char response_buffer[1024]

snprintf(response_buffer, sizeof(response_buffer),
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: application/json; charset=utf-8\r\n"
    "Content-Length: %d\r\n"
    "Connection: close\r\n"
    "\r\n"
    "%s",
    payload_len, json_payload);
```

此處設定 `Connection: close` 標頭是在簡化伺服器的實作難度，告知 Python 客戶端在接收完本

次回應後，伺服器將主動關閉該 TCP 連線，不維持 Keep-Alive 狀態。

2. 可靠的回應發送機制: `send_all()`

如同 `recv()` 無法保證一次讀完所需長度的資料，發送資料的系統呼叫 `send()` 同樣不保證一次就能將所有應用層資料推入作業系統核心的 TCP 傳送緩衝區中³。

- `ssize_t send(int sockfd, const void *buf, size_t len, int flags);` 當核心的傳送緩衝區可用空間小於 `len` 時，`send()` 可能只發送了部分資料即回傳²³。因此，實作一個能處理剩餘位元組的迴圈發送函數是不可或缺的：

```
C
int send_all(int sockfd, const char *buffer, size_t length)
{
    size_t total_sent = 0;
    while (total_sent < length)
    {
        ssize_t sent_bytes = send(sockfd, buffer + total_sent, length - total_sent, 0);
        if (sent_bytes < 0)
        {
            // 發生網路層級錯誤，應記錄並中斷傳送
            return -1;
        }
        if (sent_bytes == 0)
        {
            // 對端意外關閉
            break;
        }
        total_sent += sent_bytes; // 推進指標與計算剩餘量
    }
    return (total_sent == length ? 0 : -1);
}
```

此邏輯透過指標運算 `buffer + total_sent` 精確控制每一次重試時資料在記憶體中的起始偏移量 (Offset)，確保回應資料的位元組能完整無缺地交由核心發送²³。

六、系統層級之例外防禦: SIGPIPE、錯誤處理與網路穩定性

在生產環境的 API 服務中，網路連線的狀態充滿變數。Python 客戶端可能因為腳本崩潰、使用者強制中斷終端機，或因設定了極短的 `timeout` 參數，而在 C 伺服器尚未完成 HTTP 請求解析或回應傳送前，便已中斷 TCP 連線¹³。

1. 致命的 SIGPIPE 訊號

當客戶端意外關閉連線時，其作業系統會向 C 伺服器發送一個 RST (Reset) 封包，直接銷毀該 TCP 連線。然而，若此時 C 伺服器程式碼正執行到 `send()` 函數，嘗試將 HTTP 回應寫入這個已經

損毀的 `client_fd` 中，Linux/POSIX 作業系統會啟動其例外處理機制，向該執行緒發送 SIGPIPE (Broken Pipe) 訊號 (訊號編號為 13)⁵。

- 系統預設崩潰行為：根據 POSIX 標準，若應用程式未註冊處理 SIGPIPE 訊號的常式 (Handler)，其預設行為是立即強行終止 (Terminate) 整個處理程序¹²。這意味著只要有一個 Python 客戶端意外斷線，整個 C API 伺服器就會崩潰，導致服務全面中斷。
- 全域訊號忽略方案：最簡潔的防禦方式是在伺服器程式的 `main()` 函數開頭，呼叫 `signal(SIGPIPE, SIG_IGN);`，指示作業系統核心忽略此訊號¹²。當忽略該訊號後，`send()` 遇到損毀的連線將不再觸發中斷，而是單純回傳 -1，並將全域錯誤變數 `errno` 設定為 EPIPE，開發者即可在 `send_all()` 的回傳值檢查中優雅地處理錯誤並釋放資源¹²。
- 單次呼叫級別防禦方案：若不希望更改全域訊號處理，另一種更細緻的做法是在呼叫 `send()` 時，於最後一個 `flags` 參數傳入 MSG_NOSIGNAL 巨集。這會指示核心僅針對此次特定的寫入操作，若遭遇管線破裂，不要產生 SIGPIPE 訊號¹²。這在開發可供其他程式呼叫的函式庫時，是避免干擾主程式訊號機制的最佳實踐¹²。

2. 連線資源釋放與防範殭屍資源

無論 HTTP 請求的處理成功與否，甚至在 `recv()` 或 `send()` 過程中遭遇上述的網路中斷錯誤，伺服器都必須確保對每一個連線呼叫 `close(client_fd);`³。在 Linux 系統中，檔案描述符的總數受限於 `ulimit` 與核心設定。若遺漏了 `close()` 呼叫，每一次連線都會洩漏一個檔案描述符與相關的核心記憶體，最終將導致系統耗盡資源，觸發 EMFILE (Too many open files) 錯誤，使得伺服器無法再 `accept()` 任何新連線¹⁶。

七、結論

透過 C 語言由底向上實作一個能支援 Python requests 模組互動的 HTTP API 伺服器，是一場深入作業系統核心、記憶體管理與網路協定底層的嚴謹技術實務¹。與 Python 高度抽象的開發體驗截然不同，C 語言要求開發者擔負起與作業系統核心直接交涉的責任。

開發者必須精確掌控位址結構體 `struct sockaddr_in` 記憶體佈局與位元組對齊；深刻理解諸如 AF_INET、SOCK_STREAM 與 SO_REUSEADDR 等巨集所代表的整數意義及其在核心資源分配中的影響；並且嚴格透過 `htons()` 與 `htonl()` 維護網路字節序的和諧。

更關鍵的是，開發者必須跨越協定層級的認知陷阱，深刻體認 TCP 作為字節流協定無邊界的本質。透過精心設計的 `recv()` 與 `send()` 迴圈操作，輔以對字串記憶體的動態解析 (如運用 `strstr` 定位標頭邊界、`sscanf` 提取 Content-Length 整數值)，方能在零碎的位元組中還原出合法的 HTTP 請求。最後，透過阻擋 SIGPIPE 訊號等系統級別的防禦機制的引入，確保了伺服器在面對不可預測之網路環境時的堅韌性。掌握這些底層細節，不僅是實作高性能網路服務的基石，更是系統程式設計領域中極具價值的核心競爭力。

引用的著作

1. How to Implement a Simple HTTP Server Using POSIX Sockets in C - OneUptime, <https://oneuptime.com/blog/post/2026-03-20-c-http-server-raw-sockets/view>
2. How I Built a Simple HTTP Server from Scratch using C - DEV Community, <https://dev.to/jeffreythecoder/how-i-built-a-simple-http-server-from-scratch-using-c-739>
3. Simple C example of doing an HTTP POST and consuming the response - Stack

Overflow,

<https://stackoverflow.com/questions/22077802/simple-c-example-of-doing-an-http-post-and-consuming-the-response>

4. How to Use the struct sockaddr_in for IPv4 Addressing in C - OneUptime, <https://oneuptime.com/blog/post/2026-03-20-c-sockaddr-in-ipv4-addressing/view>
5. [unknown_url](#)
6. How to Understand struct sockaddr_in6 in C - OneUptime, <https://oneuptime.com/blog/post/2026-03-20-struct-sockaddr-in6-c/view>
7. Socket Programming in C - GeeksforGeeks, <https://www.geeksforgeeks.org/c/socket-programming-cc/>
8. setsockopt(), getsockopt() - GTA UFRJ, <https://www.gta.ufrj.br/ensino/eel878/sockets/setsockoptman.html>
9. setsockopt() — Set options associated with a socket - IBM, <https://www.ibm.com/docs/en/zos/3.1.0?topic=functions-setsockopt-set-options-associated-socket>
10. listen(2) - Linux manual page - man7.org, <https://man7.org/linux/man-pages/man2/listen.2.html>
11. Difference Between SO_REUSEADDR and SO_REUSEPORT - GeeksforGeeks, https://www.geeksforgeeks.org/linux-unix/difference-between-so_reuseaddr-and-so_reuseport/
12. How to prevent SIGPIPEs (or handle them properly) - Stack Overflow, <https://stackoverflow.com/questions/108183/how-to-prevent-sigpipes-or-handle-them-properly>
13. C signal (sigpipe, sighup) via Socket - Stack Overflow, <https://stackoverflow.com/questions/34319025/c-signal-sigpipe-sighup-via-socket>
14. difference between sockaddr_in and sockaddr : r/cpp_questions - Reddit, https://www.reddit.com/r/cpp_questions/comments/1mzsne8/difference_between_sockaddr_in_and_sockaddr/
15. The sockaddr Hierarchy, <https://web.stanford.edu/class/archive/cs/cs110/cs110.1182/autumn-2017/lectures/15-networking-socket-hierarchy.html>
16. accept(2) - Linux manual page - man7.org, <https://man7.org/linux/man-pages/man2/accept.2.html>
17. bind a name to a socket - Ubuntu Manpages, <https://manpages.ubuntu.com/manpages/jammy/man2/bind.2.html>
18. accept(2) - OpenBSD manual pages, <https://man.openbsd.org/accept.2>
19. Correct way to use SO_REUSEADDR? - Stack Overflow, <https://stackoverflow.com/questions/75160845/correct-way-to-use-so-reuseaddr>
20. How do I use setsockopt(SO_REUSEADDR)? - Stack Overflow, <https://stackoverflow.com/questions/24194961/how-do-i-use-setsockoptso-reuseaddr>
21. listen(2) — Linux manual pages - LinuxCampus.net,

- <https://www.linuxcampus.net/documentation/man-html/htmlman2/listen.2.html>
22. Socket Programming - Dartmouth Computer Science,
<https://www.cs.dartmouth.edu/~campbell/cs50/socketprogramming.html>
 23. How to Send and Receive Binary Data Over TCP Sockets in C - OneUptime,
<https://oneuptime.com/blog/post/2026-03-20-binary-data-tcp-c/view>
 24. tcp(7) - Linux manual page - man7.org,
<https://man7.org/linux/man-pages/man7/tcp.7.html>
 25. how can I read data from a POST request body, when I write a web server using C,
<https://stackoverflow.com/questions/65580275/how-can-i-read-data-from-a-post-request-body-when-i-write-a-web-server-using-c>
 26. Why recv blocks before receiving all Content-Length? - Stack Overflow,
<https://stackoverflow.com/questions/74572353/why-recv-blocks-before-receiving-all-content-length>
 27. recv() — Receive data on a socket - IBM,
<https://www.ibm.com/docs/en/zos/2.5.0?topic=functions-recv-receive-data-socket>
 28. A simple HTTP server from scratch - Trung Vuong Thien,
<https://trungams.github.io/2020-08-23-a-simple-http-server-from-scratch/>
 29. best way to read post request from a tcp socket? - Stack Overflow,
<https://stackoverflow.com/questions/75219657/best-way-to-read-post-request-from-a-tcp-socket>
 30. Guide to Python Requests Headers - Scrapfly Blog,
<https://scrapfly.io/blog/posts/python-requests-headers-guide>
 31. recv() on HTTP - CodeGuru Forums,
[https://forums.codeguru.com/showthread.php?485560-recv\(\)-on-HTTP](https://forums.codeguru.com/showthread.php?485560-recv()-on-HTTP)
 32. Parsing HTTP POST Request Body in C - Web Server with CGI Support - ProjectAI,
<https://projectai.in/projects/ef6ac0fb-43ff-4e6c-955c-5358920fe9ce/tasks/eec4caa9-ac0e-4f4f-8780-cd78af3c65fc>
 33. strstr() in C/C++ - GeeksforGeeks,
<https://www.geeksforgeeks.org/cpp/strstr-in-cpp/>
 34. strstr, <https://pubs.opengroup.org/onlinepubs/009695399/functions/strstr.html>
 35. Socket Programming - How to get recv() dynamically : r/C_Programming - Reddit,
https://www.reddit.com/r/C_Programming/comments/1pr0x6d/socket_programming_how_to_get_recv_dynamically/
 36. sscanf() — Read and Format Data - IBM,
<https://www.ibm.com/docs/en/zos/2.5.0?topic=programs-sscanf-read-format-data>
 37. How to Read Data Using sscanf() in C? - GeeksforGeeks,
<https://www.geeksforgeeks.org/c/how-to-read-data-using-sscanf-in-c/>
 38. POST request with headers and body - Python - GeeksforGeeks,
<https://www.geeksforgeeks.org/python/python-requests-post-request-with-headers-and-body/>
 39. Quickstart — Requests 2.34.2 documentation - Requests: HTTP for Humans,
<https://requests.readthedocs.io/en/latest/user/quickstart/>

40. response.headers - Python requests - GeeksforGeeks,
<https://www.geeksforgeeks.org/python/response-headers-python-requests/>
41. The Ultimate Guide - Python Requests Headers,
<https://python-requests.org/python-requests-headers/>
42. How to handle a broken pipe exception (SIGPIPE) in FIFO pipe? -
LinuxQuestions.org,
<https://www.linuxquestions.org/questions/programming-9/how-to-handle-a-broken-pipe-exception-sigpipe-in-fifo-pipe-866132/>
43. How to handle a SIGPIPE error inside the object that generated it? - Stack
Overflow,
<https://stackoverflow.com/questions/24920748/how-to-handle-a-sigpipe-error-in-side-the-object-that-generated-it>
44. send(2) - Linux manual page - man7.org,
<https://man7.org/linux/man-pages/man2/sendmsg.2.html>